

Machine Learning - 2301CS621

02-12-2025

Lab - 2

EDA & Pipeline: Google Play Store Apps

Dataset: Google Play Store Apps (Available on Kaggle)

Objective: Transform raw, messy data into clean, actionable insights using Pandas and Scikit-Learn pipelines.

Focus: Data Cleaning, String Sanitization, Advanced Imputation, Correlation, and Pipelines.

1. Setup & Initialization

Exercise 1: Import Dependencies

- Import `pandas`, `numpy`, `matplotlib.pyplot`, and `seaborn`.
- Set pandas options to display all columns (visual aid).

```
In [ ]: import pandas as pd  
import numpy as np
```

```
In [ ]: import seaborn as sns  
import matplotlib.pyplot as plt
```

Exercise 2: Data Loading & Initial Inspection

- Load the `googleplaystore.csv` file.
- Display the first 5 rows.
- **Check:** Look closely at the `Installs`, `Size`, and `Price` columns. Notice they are currently Objects (strings), not numbers.

```
In [ ]: df = pd.read_csv("googleplaystore.csv")
df.head()
```

2. Data Integrity Check

Exercise 3: Audit Data Types and Missing Values

- Use a single command to view data types (`dtypes`) and non-null counts.
- Calculate the *percentage* of missing values for each column.

```
In [ ]: df.info()
```

```
In [ ]: print(df.isnull().sum()/len(df)*100)
```

Exercise 4: Handling Duplicates

- Duplicate entries skew results. Check for duplicate rows.
- Drop duplicates, keeping the *first* occurrence. Verify the shape change.

```
In [ ]: # For Checking Duplicates use duplicated()
print("Duplicate before ",df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Duplicate After ",df.duplicated().sum())
# Use drop_duplicates to Drop()
```

```
In [ ]: df.shape
```

3. Advanced String Sanitization (Crucial Step)

Exercise 5: Cleaning the 'Installs' Column

- The `Installs` column contains characters like `+` and `,` (e.g., "10,000+").
- Remove these characters.
- Convert the column to a numeric integer type.

```
In [ ]: # use Column.astype(str).str.replace
df["Installs"].astype(str).str.replace("+","").str.replace(",","")
```

```
In [ ]: # use to_numeric to convert to int
df["Installs"] = pd.to_numeric(df["Installs"],errors="coerce")
```

```
In [ ]: df["Installs"].dtype
```

Exercise 6: Cleaning the 'Price' Column

- The `Price` column contains the `$` symbol (e.g., "\$4.99").
- Remove the symbol.
- Convert the column to a `float` .

```
In [ ]: # Same as Above
df['Price']=df['Price'].astype(str).str.replace('$', '')
```

```
In [ ]: df['Price']=pd.to_numeric(df['Price'],errors='coerce')
```

```
In [ ]: df["Price"].dtype
```

```
In [ ]: df.shape
```

Exercise 7: Complex Logic - Sanitizing 'Size'

- The `Size` column is messy. It contains 'M' (Megabytes), 'k' (kilobytes), and string 'Varies with device'.
- **Task:** Write a function (or apply lambda) to:
 1. Replace 'k' with 'e+3' and 'M' with 'e+6'.
 2. Coerce 'Varies with device' to `NaN`.
 3. Convert the string to a number.

```
In [ ]: # Hint: Define a function clean_size(x).
# Hint: If 'M' in x: return float(x.replace('M', '')) * 1000000
# Hint: Handle the 'Varies with device' edge case carefully.

def clean_size(x):
    x = str(x)
    if 'M' in x:
        # Convert 19M to 19000000
        return float(x.replace('M', ""))*1000000

    elif 'k' in x:
        # Convert 500k to 500000
        return float(x.replace('k', ""))*1000

    elif 'Varies with device' in x:
        # Handle string edge case
        return np.nan

    else:
        # Attempt to convert or return NaN
        try:
            return float(x)
        except:
            return np.nan

# Use apply Method to apply above fun
df["Size"] = df['Size'].apply(clean_size)
print("Size column cleaned and converted.")
```

```
In [ ]: df.shape
```

4. Advanced Imputation

Exercise 8: Analyzing Missing 'Rating'

- The `Rating` column has missing values.
- **Visualize** the distribution of Ratings using a Histogram or KDE plot to decide between Mean vs Median imputation.

```
In [ ]: print("Missing Ratings:", df['Rating'].isnull().sum())
```

```
In [ ]: df.shape
```

```
In [ ]: sns.histplot(df.Rating,kde=True,color='orange')
plt.xlabel('Rating')
plt.ylabel('Count')
plt.title("Distribution of App Rating")
```

Exercise 9: Group-Specific Imputation

- Fill missing `Rating` values with the **Median Rating** of the specific `Category` the app belongs to.
- *Example:* If a "Business" app is missing a rating, fill it with the median rating of all "Business" apps.

```
In [ ]: df['Rating'] = df.groupby('Category')['Rating'].transform(lambda x: x.fillna(x.median))
print("Missing Rating after imputation: ",df['Rating'].isnull().sum())
#After this use transform Method
```

Exercise 10: Drop Remaining NaNs

- For the remaining columns with minimal missing data (like `Current Ver`), simply drop the rows containing NaNs to ensure a clean dataset for correlation.

```
In [ ]: df.shape
```

```
In [ ]: # dropna
df.dropna()
```

```
In [ ]: df.shape
```

```
In [ ]: print(df.isnull().sum())
```

```
In [ ]: df.to_csv("googleplaystore_cleaned.csv")
```

5. Correlation & Visualization

Exercise 11: Correlation Matrix

- Generate a correlation matrix for the numerical columns (`Rating` , `Reviews` , `Size` , `Installs` , `Price`).

```
In [ ]: # Hint: numeric_only Use CORR
corr = df.corr(numeric_only=True)
corr
```

Exercise 12: Heatmap Visualization

- Visualize the correlation matrix using a Seaborn Heatmap.
- Annotate the values.

```
In [ ]: # Hint: sns.heatmap
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap")
```

Exercise 13: Scatter Plot Analysis

- Create a Scatter Plot to analyze the relationship between `Reviews` and `Installs` .
- **Note:** You might need to use a log scale for the axes if the data is skewed.

```
In [ ]: sns.scatterplot(x=df['Reviews'],y=df['Installs'])
plt.xscale('log')
plt.yscale('log')
plt.title('Reviews vs Installs (Log Scale)')
plt.show()
```

Exercise 14: Categorical Aggregation

- Create a Bar Plot showing the top 10 Categories by **Total Installs**.

```
In [ ]: top_cats = df.groupby('Category')['Installs'].sum().sort_values(ascending=False).head(10)
sns.barplot(y=top_cats.index,x=top_cats.values,palette='coolwarm')
plt.title("Top 10 Categories by Installs")
plt.xlabel("Installs")
plt.ylabel("Category")
plt.show()
```

6. Building a Sklearn Pipeline

Exercise 15: Preprocessing Pipeline

- Imagine you want to predict the `Rating` (Target).
- Create a Scikit-Learn `ColumnTransformer` that:
 1. StandardScales the numerical features (`Reviews` , `Size` , `Installs` , `Price`).
 2. OneHotEncodes the categorical feature (`Category` , `Content Rating`).
- Display the pipeline object.

```
In [ ]: from sklearn.compose import ColumnTransformer
        from sklearn.preprocessing import StandardScaler, OneHotEncoder

        num_features = ['Reviews', 'Size', 'Installs', 'Price']
        cat_features = ['Category', 'Content Rating']

        preprocessor = ColumnTransformer(
            transformers=[
                ('num', StandardScaler(), num_features),
                ('cat', OneHotEncoder(), cat_features)
            ]
        )

        print(preprocessor)
```